UNIT - 5

AGILE PROCESS MODEL

Introduction to Agile

- Agile Software Development is a software development methodology that values flexibility, collaboration, and customer satisfaction. It is based on the Agile Manifesto, a set of principles for software development that prioritize individuals and interactions, working software, customer collaboration, and responding to change.
- Agile Software Development is an iterative and incremental approach to software development that emphasizes the importance of delivering a working product quickly and frequently.
- It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.

12 Principles of Agile Software Development

Agile methodologies are guided by the Agile Manifesto, which was created in 2001 by a group of software developers who wanted to find a more effective way to develop software.

1. *Customer Satisfaction Through Early and Continuous Delivery of Valuable Software:* Agile prioritizes delivering working software frequently (often in weeks), ensuring that the customer receives ongoing value.
2. *Welcome Changing Requirements:* Even late in development, Agile welcomes changes in requirements, which is seen as an opportunity to add value to the customer.
3. *Deliver Working Software Frequently:* Working software is delivered in short, iterative cycles (typically 1- 4 weeks).
4. *Business and Developers Must Work Together Daily:* Collaboration between business stakeholders and development teams is crucial for success. This daily communication ensures alignment and shared understanding.
5. *Build Projects Around Motivated Individuals:* Empowering teams with the right skills, tools, and environments to succeed is key.
6. *Face-to-Face Communication is the Most Effective:* Agile encourages teams to communicate face-to-face as much as possible, ensuring clarity and reducing misunderstandings.
7. *Working Software is the Primary Measure of Progress:* Rather than focusing on documentation or meetings, Agile measures success by the delivery of working software.
8. *Sustainable Development:* Agile promotes sustainable development, meaning that the pace of work should be balanced to avoid burnout and maintain long-term productivity.
9. *Continuous Attention to Technical Excellence:* Agile teams are encouraged to maintain a high level of technical skill and quality to enhance agility and reduce technical debt.
10. *Simplicity:* Agile emphasizes simplicity in design and development, focusing on what is necessary and eliminating unnecessary work.
11. *Self-Organizing Teams:* Agile trusts that the team knows how to best achieve the goals, allowing them to self-organize and make decisions autonomously.

12. *Regular Reflection and Adjustment:* Teams should reflect on how to become more effective and adjust their processes accordingly.

Popular Agile Methodologies

While Agile itself is a broad philosophy, several specific frameworks and methodologies have been developed to implement Agile principles. Some of the most popular ones include:

Scrum

- Scrum is one of the most widely used Agile frameworks. It focuses on iterative development through fixed-length iterations called sprints (usually 2-4 weeks).
- Scrum roles include Product Owner (who defines features and priorities), Scrum Master (who facilitates the process), and Development Team (who builds the product).
- Key Scrum ceremonies include Sprint Planning, Daily Standups (or Daily Scrum), Sprint Review, and Sprint Retrospective.

Kanban

- Kanban is a visual method for managing work, where tasks are represented on a board (often divided into columns such as "To Do," "In Progress," and "Done").
- Unlike Scrum, Kanban does not prescribe specific roles or sprints. It focuses on continuous delivery and managing work in progress by limiting the number of tasks in each column (Work In Progress limits).

Extreme Programming (XP)

- Extreme Programming (XP) is an Agile software development methodology that focuses on high-quality code, customer satisfaction, and rapid response to changing requirements through frequent feedback and short development cycles.

- Extreme Programming Key Practices are:
    - *Pair Programming*: Two developers work together at one workstation
    - *Test-Driven Development (TDD)*: Write tests before writing code
    - *Continuous Integration* – Code is integrated and tested frequently
    - *Small Releases* – Deliver working software in short iterations
    - *Refactoring* – Improve code structure without changing functionality
    - *Simple Design* – Do only what is needed today
    - *Collective Code Ownership* – Anyone can improve any part of the code
    - *On-site Customer* – Customer is available for constant feedback
    - *40-hour Work Week* – Avoid developer burnout

Lean Software Development

- Lean Software Development (LSD) is an Agile methodology derived from Lean manufacturing (Toyota Production System) that focuses on maximizing customer value while minimizing waste.
- It emphasizes improving efficiency by reducing delays, improving quality, and making continuous improvements.
- Core Principles of Lean Software Development are:
    - *Eliminate Waste***:** Remove unnecessary features, delays, rework, and documentation
    - *Build Quality In:* Prevent defects rather than fixing them later

- *Create Knowledge:* Learn continuously through feedback and experimentation
- *Defer Commitment:* Make decisions as late as possible when more information is available
- *Deliver Fast:* Short cycles and quick releases.
- *Respect People:* Empower teams and encourage collaboration
- *Optimize the Whole:* Focus on the entire value stream, not just parts

Benefits of Agile

- *Customer Satisfaction:* Agile encourages frequent releases of working software, meaning that customers receive updates and new features regularly. This leads to higher customer satisfaction.
- *Flexibility and Adaptability:* Agile allows teams to adapt to changing requirements, which is crucial in environments where the business or technology landscape changes quickly.
- *Faster Time to Market:* Agile practices enable teams to deliver smaller, incremental pieces of the project frequently, reducing the time taken to launch the product.
- *Improved Product Quality:* With practices like test-driven development (TDD), continuous integration, and regular feedback, Agile teams can ensure higher product quality and fewer defects.
- *Collaboration:* Agile promotes collaboration between all stakeholders, including developers, customers, and business users, ensuring that everyone is aligned on project goals.
- *Risk Reduction:* Through iterative cycles and constant feedback, potential issues are identified and addressed early in the process, reducing the likelihood of project failure.

Examples of Agile Software Development

A Software company named ABC wants to make a new web browser for the latest release of its operating system. The deadline for the task is 10 months. The company's head assigned two teams named Team A and Team B for this task. To motivate the teams, the company head says that the first team to develop the browser would be given a salary hike and a one-week full-sponsored travel plan. With the dreams of their wild travel fantasies, the two teams set out on the journey of the web browser. Team A decided to play by the book and decided to choose the Waterfall model for the development. Team B after a heavy discussion decided to take a leap of faith and choose Agile as their development model.

The Development Plan of the Team A is as follows:

- Requirement analysis and Gathering – 1.5 Months
- Design of System – 2 Months
- Coding phase – 4 Months
- System Integration and Testing – 2 Months
- User Acceptance Testing – 5 Weeks

The Development Plan for the Team B is as follows:

- Since this was an Agile, the project was broken up into several iterations.
- The iterations are all of the same time duration.
- At the end of each iteration, a working product with a new feature has to be delivered.

- Instead of Spending 1.5 months on requirements gathering, they will decide the core features that are required in the product and decide which of these features can be developed in the first iteration.
- Any remaining features that cannot be delivered in the first iteration will be delivered in the next subsequent iteration, based on the priority.
- At the end of the first iterations, the team will deliver working software with the core basic features.

The team has put their best efforts into getting the product to a complete stage. But then out of the blue due to the rapidly changing environment, the company's head came up with an entirely new set of features that wanted to be implemented as quickly as possible and wanted to push out a working model in 2 days.

Team A was now in a fix, they were still in their design phase and had not yet started coding and they had no working model to display.

Moreover, it was practically impossible for them to implement new features since the waterfall model there is not revert to the old phase once you proceed to the next stage, which means they would have to start from square one again. That would incur heavy costs and a lot of overtime.

Team B was ahead of Team A in a lot of aspects, all thanks to Agile Development. They also had a working product with most of the core requirements since the first increment. And it was a piece of cake for them to add the new requirements. All they had to do was schedule these requirements for the next increment and then implement them.

SCRUM

- Scrum is an agile framework used primarily in software development but applicable to various types of project management. It emphasizes teamwork, accountability, and iterative progress.
- Scrum is the dominant Agile software development framework, used by **87%** of organizations adhering to Agile methodologies according to statistics gathered by Scrum.org.

Key Aspects of SCRUM

Scrum Roles

The following are key roles in Scrum projects.

- **Product Owner**
  - This can only be one person, not a committee, as multiple product owners tend to confuse team priorities.
  - The product owner is responsible for maximizing the business value delivered by the team, prioritizes the backlog, accepts or rejects work, and helps define what "done" means during the project.
  - The product owner should be knowledgeable, empowered, and engaged.

- **Scrum Master**
  - Facilitates the Scrum process, ensuring the team follows Scrum practices and removing any obstacles or impediments that could hinder progress.

    - The Scrum Master is often described as a servant leader - not commanding or controlling.

- **Team**
    - The people responsible for turning the product backlog items into increments of value during each Sprint.
    - A team generally includes Business Analysts, Developers, and Testers.
    - Teams are cross-functional and can consist of 7 to 9 members.
    - Teams come together as one cross functional team and work together on the backlog. Teams are completely focused on quality.

Scrum in Action

- **Sprint Planning**
    - A Sprint is where Scrum teams work to complete a set amount of work in a fixed time period.
    - During Sprint planning the Scrum Master, Product Owner, and the Team finalize what is going to take place in the current Sprint.
    - User stories are taken from the backlog, a prioritized list of tasks needed to accomplish the planned work, to build the Sprint.
    - Sprint planning takes place no later than the first day of each Sprint and takes approximately two hours per each week of the Sprint.
    - For example, Sprint planning for a two-week Sprint could be expected to take four hours to complete.

- **Product Backlog Refinement**
    - The processes where the Scrum Master, Product Owner, Stakeholder, and the Team add details and estimates to the backlog.
    - Product backlog refinement takes place as often as necessary during the Sprint but should take no more than 10 percent of the Sprint's duration.

- **Daily Scrum (or Standup)**
    - A quick meeting of approximately 15 minutes that anyone can attend, but where only the Scrum team can talk.
    - These meetings are held at the same time and place each day while the Sprint is in progress to create a work plan for the next 24 hours.
    - Each team member answers three questions: What did you do yesterday? What will you do today? Do you have any obstacles?

- **Sprint Review (Demo)**
    - A meeting at the end of the Sprint, where the Scrum Master, Product Owner, Stakeholder, and the Team, review the work that has been completed to determine whether the product meets the definition of "done".
    - Sprint reviews typically take one to two hours**,** depending on the size of the Sprint.

- **Sprint Retrospective**
    - A meeting that takes place after the Sprint review and before the next Sprint planning meeting.

- Retrospectives feature the Scrum Master, Product Owner, and the Team discussing (without finger-pointing) what went well and what could go better in the future.
- Sprint retrospectives take approximately 45 minutes for each week of the Sprint.

Communications Tools Commonly Used in Scrum

- **Product Backlog**
  - An ordered list of features, enhancements, bug fixes, and technical work required for the product. It is maintained by the Product Owner and is dynamic, constantly evolving.

- **User Stories**
  - Descriptions of software features, written from a user-perspective, which follow a common format.
  - As a [type of user], I want [some goal] so that [I can achieve some benefit].

- **Sprint Backlog**
  - A subset of the Product Backlog selected for a specific sprint. It includes tasks that the team commits to complete during the sprint.

- **Release Burndown Chart**
  - Visually shows the progress of the Sprint.

Example: User Registration

User Story - *As a* new user, *I want* to register for an account *so that* I can access personalized content and save my preferences.

Acceptance Criteria:
- User can input a username, email address, and password.
- User receives a confirmation email after registering.
- The account is successfully created when the user confirms their email address.

Example: Search Functionality

User Story - *As a* shopper, *I want* to be able to search for products by keyword *so that* I can quickly find the items I want to buy.

Acceptance Criteria:
- User can enter a search term in the search bar.
- Search results are displayed in a grid with product names and images.
- Results are ranked by relevance or category.

Example: Password Reset

User Story - *As a* user who has forgotten their password, *I want* to reset my password *so that* I can regain access to my account.

Acceptance Criteria:

- User can click on a "Forgot Password" link on the login page.
- User receives an email with a password reset link.
- User can successfully reset their password through the link.

Scrum Process Flow

The Scrum process flow outlines how Scrum practices and roles come together to deliver value in iterative cycles (Sprints).

Here's an overview of the Scrum process flow from start to finish:

1. Product Backlog → 2. **Sprint Planning** → 3. **Sprint Execution** → 4. **Daily Scrum** → 5. **Sprint Review** → 6. **Sprint Retrospective** → 7. **Next Sprint Planning** (Repeat)

Product Backlog Creation

- The Product Owner creates and maintains the Product Backlog, which is an ordered list of features, enhancements, bug fixes, technical work, and other tasks necessary to deliver a product.
- The Product Backlog is dynamic, continuously evolving as the product grows and the team's understanding of the requirements changes

Sprint Planning

- At the start of each Sprint, the Sprint Planning meeting takes place.
- The Scrum Team (including the Product Owner, Scrum Master, and Development Team) participates in planning what will be worked on in the upcoming Sprint.
    - The Product Owner explains the highest priority items from the Product Backlog.
    - The Development Team selects the items from the Product Backlog they can commit to delivering during the Sprint.
- The team then defines how the work will be accomplished and creates the Sprint Backlog, which contains the tasks needed to complete the selected backlog items.

Sprint Execution

- The team starts working on the Sprint Backlog items in a time-boxed Sprint (usually 1–4 weeks).
- Daily Scrums (stand-up meetings) are held every day during the Sprint to track progress, synchronize activities, and identify obstacles or issues.
- The Development Team collaborates and organizes themselves to meet the Sprint Goal and ensure the tasks are completed by the end of the Sprint.

Work In Progress and Adaptation

- During the Sprint, the team may inspect and adapt their approach to ensure that the work is progressing smoothly. They may adjust based on new information, technical challenges, or feedback.
- The Scrum Master helps remove impediments that may be hindering the team's progress.

Sprint Review

- At the end of the Sprint, a Sprint Review meeting is held.
    - The Development Team demonstrates the completed product increment.
    - The Product Owner reviews the increment and assesses whether it meets the definition of "Done."
    - The Product Backlog may be updated based on feedback and changes in business priorities.

Sprint Retrospective

- After the Sprint Review, the team holds a Sprint Retrospective.
    - The team reflects on the Sprint process itself (not the product).
    - The team discusses what went well, what didn't, and how they can improve in the next Sprint.
    - Action items are created to improve the team's performance or the Scrum process for the next Sprint.

Repeat (Next Sprint)

- The process repeats with a new Sprint Planning meeting for the next Sprint.
- Any new items added to the Product Backlog are refined, and the team prepares for the next Sprint.

Challenges of Agile

- ***Lack of Documentation:*** Since Agile focuses on working software over comprehensive documentation, some teams may struggle to produce sufficient documentation, which can create challenges later in the project.

- ***Requires Experienced Teams:*** Agile works best with self-organizing, cross-functional teams. Without the right skill sets and experience, teams may struggle with Agile's more flexible and autonomous nature.

- ***Stakeholder Engagement:*** Agile requires frequent input and collaboration from stakeholders. If stakeholders are not available or engaged, the project can lose focus and direction.

- ***Scope Creep****:* If not managed carefully, Agile's flexibility can lead to scope creep, as new features and requirements are continuously added.

Scope creep (sometimes known as "requirement creep" or even "feature creep") refers to how a project's requirements increase over the project lifecycle.

Burn Down Chart

- A Burn Down Chart is a graphical representation used in project management, particularly in Agile frameworks like Scrum, to track the progress of work remaining in a project or sprint over time.
- It visually shows how much work is left to complete (usually measured in hours, story points, or tasks) versus the time remaining in the project or sprint.

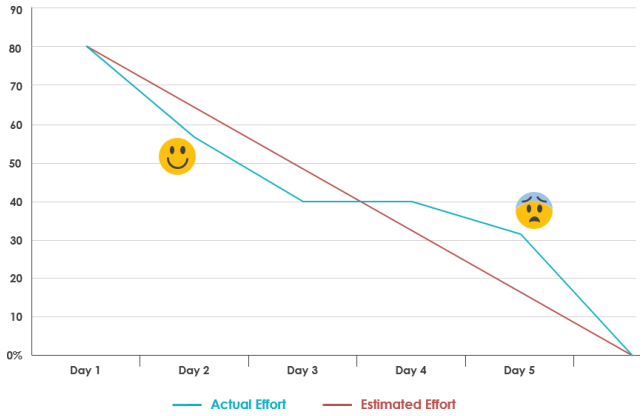
Key Components of a Burn Down Chart

- ***X-axis (horizontal axis):*** Represents the time, usually in days or iterations (e.g., Sprint days).

- ***Y-axis (vertical axis):*** Represents the amount of work remaining**,** which could be measured in units like story points, tasks, or hours.

- ***Ideal Line:*** This is a diagonal line that represents the ideal progress where work is completed at a consistent rate. It starts at the total amount of work at the beginning of the project or sprint and ends at zero when the work is completed.

- ***Actual Progress Line:*** This is a line that reflects the actual progress of the project. It shows the remaining work at the end of each day or sprint iteration, based on the team's work completed.



- At the beginning of the sprint or project, the total work is plotted at the top of the Y-axis.
- Every day, the team updates the chart to show how much work remains.
- The ideal line descends steadily from the total work to zero.
- The actual progress line shows how the team is performing. If the team is on track, the actual line will closely follow the ideal line. If it's above the ideal line, it means the team is behind, and if it's below, the team is ahead of schedule.

Benefits

- ***Transparency:*** The Burn Down Chart provides a clear and visual representation of the progress of work, making it easy for teams and stakeholders to see if the project is on track.
- ***Motivation:*** It helps team members see their progress and may encourage them to keep working toward the goal.
- ***Early Identification of Issues:*** If the actual progress line is consistently above the ideal line, it indicates that there are issues, such as scope creep, delays, or unforeseen complications, and corrective actions can be taken early.

- ***Forecasting:*** By analyzing the trend of the actual progress line, teams can predict whether they are likely to complete the work by the deadline and adjust their efforts if necessary.

Types of Burn Down Charts

- ***Sprint Burn Down Chart:*** Tracks the progress of work during a single sprint.
- ***Release Burn Down Chart:*** Tracks the progress of work for a release or larger project across multiple sprints.

Example

Imagine a Scrum team has 100 story points of work to do over a 10-day sprint. Each day, they update the burn down chart to show how many points remain. By the end of day 10, ideally, the line should hit zero, indicating that all the work has been completed.

SAFe (Scaled Agile Framework)

- SAFe (Scaled Agile Framework) is a set of principles, practices, and competencies that enable organizations to scale agile development practices across large teams, departments, or even entire enterprises.
- It provides a structured approach for applying agile principles to complex, large-scale projects while ensuring alignment, collaboration, and delivery of value across all levels of the organization.

Key Components of SAFe

1. ***Principles:*** SAFe is built around several guiding principles derived from agile development, lean thinking, and systems thinking. These principles help organizations move towards better agility and continuous improvement.

2. ***Layers:*** SAFe introduces multiple levels of planning and execution to accommodate large teams:
    - *Team Level:* Focuses on delivering value within individual teams using Scrum or Kanban.
    - *Program Level:* Ensures that multiple agile teams work together to deliver value. This level uses the concept of Agile Release Trains (ARTs), which are teams of teams aligned to a common mission.
    - *Portfolio Level:* Manages strategy, investments, and the flow of value across the organization. It ensures that the work being done aligns with the organization's goals.
    - *Large Solution Level:* Applied in scenarios where multiple ARTs work together on larger and more complex solutions.

3. ***Agile Release Trains (ARTs):*** These are virtual teams of teams that plan, commit, and execute together to deliver continuous value. ARTs align teams to common goals and

cadence, typically working in Program Increments (PIs), which are similar to agile sprints but occur over a longer time frame (8–12 weeks).

4. ***Program Increments (PIs):*** A program increment is a time-boxed iteration (typically 8–12 weeks) in which an ART delivers incremental value. PIs include multiple iterations or sprints, and their success is measured by the ability to deliver business value.

5. ***Roles:*** SAFe includes various roles designed to support the scaled agile process:
    - *Release Train Engineer (RTE):* Acts as a servant leader and facilitator for the ART.
    - *Product Owner and Product Manager:* Ensure the teams are focused on delivering high-value features and capabilities.
    - *System Architect:* Defines the technical architecture and ensures the system is scalable and sustainable.

6. ***Value Stream:*** SAFe emphasizes optimizing the flow of value across the entire organization, not just within individual teams. This is done by organizing teams and initiatives around value streams, which represent the steps required to deliver a product or service.

7. ***Cadence and Synchronization:*** Regular, consistent planning, reviews, and retrospectives across the teams and ARTs are a critical part of SAFe. The cadence ensures that teams and stakeholders align their work, providing predictable rhythms for delivery.

Scrum@Scale

- Scrum@Scale is a framework for scaling Scrum to larger, more complex organizational structures.
- It was created by Jeff Sutherland, one of the co-creators of Scrum, and is designed to extend Scrum principles and practices beyond individual teams to the entire organization.
- Scrum@Scale provides a flexible, lightweight structure that can be applied to organizations of any size, enabling them to scale Scrum effectively and deliver value across multiple teams while maintaining the core principles of Scrum.

Key Components of Scrum@Scale

1. ***The Scrum Master Cycle:*** The Scrum Master Cycle focuses on scaling the role of the Scrum Master across the entire organization to facilitate the effective use of Scrum at all levels. This cycle is responsible for removing impediments and ensuring that Scrum teams can work efficiently within the larger system.

    - *Scrum of Scrums:* At the team level, Scrum Masters participate in a Scrum of Scrums meeting to coordinate the work of multiple Scrum teams. This meeting typically happens at the start of each sprint to discuss dependencies, impediments, and progress toward the sprint goal.

- *Meta-Scrum:* This is a group of Scrum Masters (or their representatives) who are responsible for ensuring the alignment and scaling of Scrum practices across the organization. The Meta-Scrum works to continuously improve the process and remove obstacles across teams.
- *Scaling Scrum Masters:* As the organization grows, more Scrum Masters are added to facilitate coordination between multiple teams, ensuring that Scrum practices are standardized and improving the overall Scrum process.

2. ***The Product Owner Cycle:*** The Product Owner Cycle focuses on scaling the Product Owner role to manage the product backlog across multiple teams. This cycle ensures that the product backlog is effectively managed, refined, and prioritized across the entire organization.

    - *Product Owner Network:* This is a group of Product Owners who are responsible for managing different aspects of the overall product or solution. These Product Owners are tasked with maintaining a cohesive vision and ensuring that the work done by all Scrum teams aligns with the strategic goals.
    - *Backlog Refinement:* Product Owners work together to ensure that there is one unified and prioritized backlog that reflects the needs of the business, customers, and stakeholders.
    - *Chief Product Owner:* In larger organizations, a Chief Product Owner may be established to coordinate and oversee the work of multiple Product Owners, ensuring alignment and prioritization at the portfolio level.

3. ***The Scrum@Scale Framework:*** Scrum@Scale uses two main cycles—the Scrum Master Cycle and the Product Owner Cycle—which are interconnected to help scale Scrum.

    - *Team Level:* Scrum@Scale starts with Scrum at the team level, where cross-functional Scrum teams work together in sprints to deliver incremental value.
    - *Scrum of Scrums:* Teams at the next level work together in a Scrum of Scrums, a meeting where Scrum Masters from different teams meet to coordinate dependencies and resolve challenges across teams.
    - *Scaled Product Backlog:* The backlog is divided across multiple Product Owners, each of whom is responsible for part of the backlog but collaborates to ensure the product vision is coherent.
    - *Executive Action Team:* The Executive Action Team (EAT) is responsible for supporting the Scrum@Scale adoption at an organizational level. This team ensures that the framework is being effectively implemented and is aligned with the company's strategic goals.

4. ***Governance and Reporting***
    - Scrum@Scale emphasizes empowerment and decentralized decision-making, which means that the framework does not require traditional top-down governance. Instead, it encourages self-managing teams and fosters collaboration and continuous improvement.
    - Reporting is aligned with the Scrum framework and is based on transparency, inspection, and adaptation.
    - Scrum@Scale advocates for metrics that focus on delivering value rather than traditional project metrics like time, cost, and scope.

Key Roles in Scrum@Scale

- ***Scrum Master:*** The Scrum Master focuses on helping teams adhere to Scrum practices, facilitating Scrum ceremonies, and removing impediments. At scale, Scrum Masters collaborate in the Scrum of Scrums and work to ensure that teams can operate smoothly within a larger system.

- ***Product Owner:*** The Product Owner is responsible for managing the backlog and ensuring that the team is working on the most valuable features. In Scrum@Scale, Product Owners collaborate within a network to align priorities and manage the backlog across multiple teams.

- ***Chief Scrum Master:*** In larger implementations, a Chief Scrum Master can be designated to coordinate and facilitate Scrum Masters across multiple teams and ensure that Scrum practices are consistently applied at scale.

- ***Chief Product Owner:*** Similarly, a Chief Product Owner may oversee the work of multiple Product Owners and ensure the alignment of the product vision across the entire organization.